Assignment 3 report: Bridge pattern

Assignment Title: Bridge Pattern Implementation — Topic B (Notifications System)

Student Name: Sanzhar Abayev

Group: SE-2538

Topic Letter: Option B (Notifications)

1. System Overview & Bridge Pattern Rationale

1.1 The Two Independent Dimensions

In this application, the domain space consists of two independent, orthogonal dimensions of variation:

1. Abstraction Dimension (Notification Types): Represents business domain abstractions that define notification rules and messaging context (e.g., standard Reminder vs. high-priority UrgentAlert).
2. Implementor Dimension (Delivery Channels): Represents low-level execution mechanisms responsible for dispatching payloads through specific mediums (e.g., EmailChannel, SmsChannel, and PushChannel).

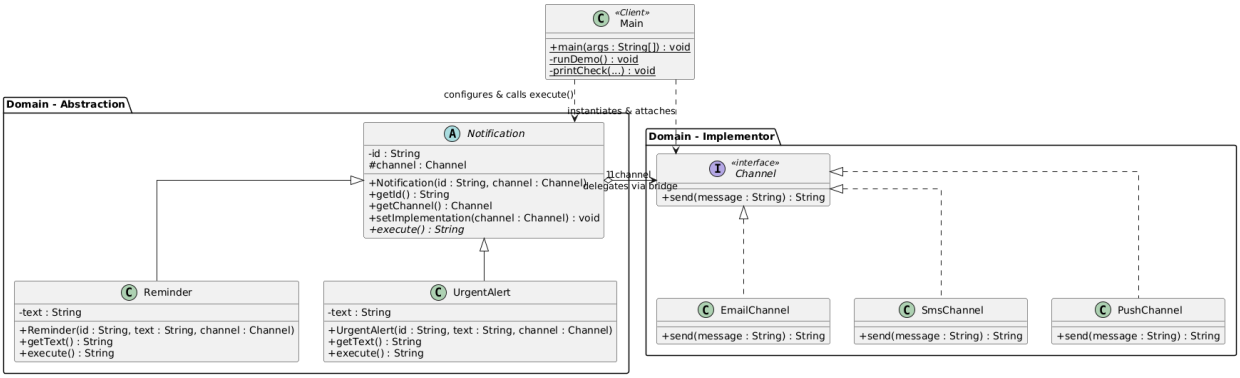
1.2 Why the Bridge Pattern Fits

Without the Bridge pattern, combining N notification types with M delivery channels using class inheritance leads to class explosion ($N \times M$ subclasses such as EmailReminder, SmsReminder, EmailUrgentAlert, SmsUrgentAlert, etc.).

By applying the Bridge pattern:

- We decouple the abstraction hierarchy (Notification) from the implementation hierarchy (Channel) via composition.
- The base class Notification maintains a reference to the Channel interface (the "Bridge").
- Adding a new notification type or delivery channel requires only 1 new class instead of N or M new subclasses, keeping system complexity linear ($N + M$).

2. PlantUML Class Diagram Code



3. Five Annotated Clean Code Excerpts

Excerpt 1: Separation of Responsibilities

```
// Abstraction delegates delivery mechanics to the Implementor interface
@Override
public String execute() {
    return channel.send("URGENT: " + text);
}
```

Explanation: UrgentAlert handles domain-specific context modification (prepending "URGENT:"), while channel.send(...) independently manages how that payload is formatted for transport.

Benefit: Ensures High Cohesion and Single Responsibility Principle (SRP).

Excerpt 2: Meaningful Names

```
public interface Channel {
    String send(String message);
}

public class EmailChannel implements Channel { ... }
```

Explanation: Names clearly express domain intent without cryptic abbreviations or technical clutter. Channel clearly defines the contract, while EmailChannel denotes the specific implementation medium.

Benefit: Improves code readability, self-documentation, and maintainability.

Excerpt 3: Small, Focused Methods

```
public void setImplementation(Channel channel) {
    this.channel = channel;
}
```

Explanation: The setImplementation method performs exactly one clear action: updating the internal Channel reference at runtime.

Benefit: Facilitates straightforward unit testing, reduces cognitive overhead, and minimizes side effects.

Excerpt 4: Avoidance of Duplication

```
public abstract class Notification {
    private final String id;
    protected Channel channel;

    public Notification(String id, Channel channel) {
        this.id = id;
        this.channel = channel;
    }

    public String getId() { return id; }
    public Channel getChannel() { return channel; }
    public void setImplementation(Channel channel) { this.channel = channel; }
}
```

Explanation: Common state (id) and bridge infrastructure (channel reference and dynamic setter) are centralized in the abstract base class Notification rather than duplicated across Reminder and UrgentAlert.

Benefit: Upholds DRY (Don't Repeat Yourself) principle and reduces maintenance surface area.

Excerpt 5: Encapsulation of State

```
public class Reminder extends Notification {
```

```
    private final String text;

    public Reminder(String id, String text, Channel channel) {
        super(id, channel);
        this.text = text;
    }

    public String getText() {
        return text;
    }
}
```

Explanation: All instance variables (id in base class, text in subclass) are marked private (or protected for bridge accessibility) and final. State modification is controlled strictly through public accessor methods or explicit operations.

Benefit: Ensures immutability of domain payload attributes, preventing illegal state mutations from external callers.

4. Extension Mechanics, Pattern Comparison & References

4.1 Extension Step Mechanics (Adding I3: PushChannel)

To fulfill step 4 of the assignment, PushChannel (I3) was introduced to the project:

1. Added Source File: Created src/domain/PushChannel.java implementing Channel.
2. Updated Client Verification: Added test cases T6 and T7 in Main.java and updated sources.txt.
3. What Remained Unchanged: The abstract base Notification, concrete abstractions Reminder and UrgentAlert, interface Channel, and existing implementors (EmailChannel, SmsChannel) remained completely untouched.
4. Compliance: Verified via extension.diff, which confirmed zero diff modifications to pre-existing domain classes, satisfying the Open-Closed Principle (OCP).

4.2 Trade-off of the Bridge Pattern

- Increased Indirection & Complexity: Introducing an extra interface layer and object composition increases the overall initial setup cost and number of classes in simple systems. Tracing runtime execution requires jumping through interface references rather than direct method calls.

4.3 Intent Comparison: Bridge vs. Adapter Pattern

- Bridge Pattern (Design-Time Strategy): Designed *upfront* before system implementation to allow abstractions and implementations to vary independently without coupling them together.
- Adapter Pattern (Legacy / Retrofit Strategy): Applied *after the fact* to integrate pre-existing, incompatible interfaces without altering their underlying source code.

5. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Fowler, M. (2006). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.